

WILL-WALLET WHITE PAPER

The Will-Wallet Inheritance Plan

*A non-custodial framework for planning, timing, and distributing
Bitcoin inheritance*

CORE PROMISE The owner retains control. Beneficiaries receive a pre-authorized path. A service provider never receives a working key combination.

Yilak Kidane

will-wallet.com

September 2026 | Product and protocol white paper

Document status and important notice

PRODUCT STATUS The Inheritance Plan Builder described here is an agreed interface specification awaiting implementation. It reorganizes two existing Will-Wallet capabilities - time-locked address generation and transaction construction - into one guided planning experience. The paper distinguishes current primitives from planned product behavior.

This paper explains a technical and product framework. It is not legal, tax, fiduciary, investment, or financial advice. Will-Wallet is designed to operate alongside a valid will, trust, beneficiary designation, and qualified professional advice; it is not a substitute for those instruments. Bitcoin transactions are irreversible after confirmation, and mistakes in key custody, script construction, fee planning, or distribution handling may permanently destroy access to funds.

All product descriptions are subject to implementation testing, independent security review, regulatory analysis, and jurisdiction-specific legal review before production use. Examples are illustrative and should not be treated as production transaction instructions.

Abstract

Bitcoin enables direct ownership without a bank, custodian, or recovery desk. That strength creates an inheritance problem: if the owner dies or becomes unable to act without transferring a valid spending path, the asset may remain visible on-chain yet permanently inaccessible. Conventional answers often require sharing a seed phrase, entrusting a custodian with a working key, or relying on a legal document that cannot itself execute a Bitcoin transaction.

Will-Wallet takes a different approach. The owner places funds in a threshold-controlled vault, prepares a distribution transaction whose outputs and amounts are fixed, signs it only partway, and gives beneficiaries or an approved monitoring process the information required to complete it at the intended time. The owner keeps unilateral control before distribution by retaining the threshold needed to re-spend the vault. Optional transaction-level timing controls when the distribution may confirm; optional output-level timelocks control when each beneficiary may spend individual portions.

The proposed Inheritance Plan Builder presents this protocol as an estate-planning workflow rather than a transaction editor. The owner works with names, portions, dates, and plain-language outcomes. Addresses, locktimes, redeem scripts, fees, and half-signed transaction artifacts are produced by the system and exposed only when needed for verification. The result is a non-custodial inheritance plan that is understandable to the owner today and actionable by a non-technical beneficiary in the future.

Contents

- 1. The inheritance gap
- 2. Design goals and product principles
- 3. The Inheritance Plan Builder
- 4. Protocol architecture and custody model
- 5. Two independent timing layers
- 6. Revocation, updating, and the irreversible boundary
- 7. Security, trust, and operational controls
- 8. Privacy and beneficiary usability
- 9. Institutional and commercial model
- 10. Implementation path and acceptance criteria
- 11. Limitations and legal positioning
- 12. Conclusion
- Appendix A. Technical requirements
- Appendix B. References

Executive summary

Will-Wallet is a Bitcoin inheritance system built around a simple boundary: planning can be understandable and assisted without making the software provider, estate firm, or monitoring service a custodian. The protocol does this by separating key custody, release timing, and beneficiary vesting into different responsibilities.

Question	Will-Wallet answer
Who controls the Bitcoin while the owner is active?	The owner. In the base 2-of-3 vault, the owner retains two keys and can spend or revise without another party.
What does a beneficiary or monitoring party receive?	A half-signed distribution transaction and supporting scripts/instructions - not a working owner key combination.
How is a known future date enforced?	The specific distribution transaction uses nLockTime with non-final input sequence values so it cannot confirm before the selected time or block.
How are beneficiary-specific schedules enforced?	Each portion can pay to a CHECKLOCKTIMEVERIFY-gated output, making that portion unspendable until its own date or block.
Can the owner change the plan?	Yes, until the distribution transaction confirms. Spending the vault with a newer owner-authorized transaction invalidates earlier half-signed copies.
What remains trusted for an unknown event?	Only detection and delivery timing. A check-in service or estate institution may release an inert artifact, but it never holds enough keys to move funds.

What the product adds

The underlying cryptographic tools already exist in Will-Wallet. The planned Builder makes them usable as one coherent plan:

- Load an existing funded vault and show the available balance.
- Choose a plan-wide release condition: a known date/block or a separately configured monitored event.
- Add beneficiaries by name, provide each beneficiary public key, and allocate BTC and percentages together.
- Create immediate, date-based, or block-height portions for each beneficiary, with derived addresses generated live.
- Review the plan as both a list and a timeline before building anything.
- Build one half-signed distribution transaction and produce a beneficiary-specific distribution package.

Saving preserves a local draft. Building creates the half-signed transaction artifact. Broadcasting and confirmation are separate later events.

1. The inheritance gap

Bitcoin removes the institution that would normally verify an heir and restore access. The network does not know who died, who was named in a will, or whether a court appointed an executor. It recognizes only valid transactions satisfying the conditions of the coins being spent.

That produces three common but unsatisfactory inheritance patterns:

1. Direct key sharing. The owner gives a seed phrase or private key to another person. This is easy to understand, but it gives the recipient present-day power over the funds and creates theft, coercion, copying, and accidental-loss risk.
2. Custodial inheritance. A company or institution holds a working key or recoverable shard. This may improve convenience, but it creates an ongoing counterparty, security, insolvency, and regulatory dependency.
3. Legal instructions without an executable path. A will or trust identifies the asset and beneficiary, but the executor may still lack the keys, scripts, device knowledge, or transaction artifact needed to act.

Will-Wallet is designed as a fourth pattern: create a technically executable inheritance path in advance while keeping unilateral spending control with the owner until the plan is actually distributed on-chain.

THE DESIGN PROBLEM A useful inheritance system must make a future transfer executable without making the future recipient - or an intermediary - a present-day custodian.

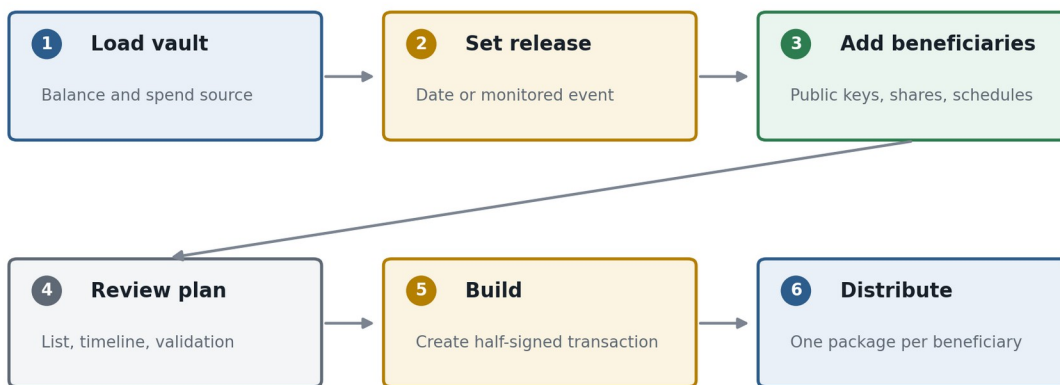
2. Design goals and product principles

Owner control The owner keeps the threshold required to spend and revise the vault before distribution confirms.	No provider custody Will-Wallet, an estate firm, or a monitoring service never receives enough keys to move funds.
Explicit timing A known date is enforced by Bitcoin; an unknown real-world event is handled by a separately disclosed monitoring process.	Per-beneficiary scheduling Each beneficiary can receive one or more portions on independently selected dates or block heights.
Human-first planning The primary interface uses beneficiaries, portions, dates, and outcomes rather than scripts and transaction fields.	Transparent artifacts Derived addresses, scripts, and transaction details remain inspectable and exportable for independent verification.
Clear irreversibility The product distinguishes editing, building, broadcasting, and confirmation, and warns before the owner crosses an irreversible boundary.	Local privacy Plan labels and draft structure remain local to the device or exported file and are not written to the blockchain.

3. The Inheritance Plan Builder

The planned Builder is not a new consensus mechanism and does not replace the current transaction engine. It is a guided arrangement of two existing Will-Wallet functions: the time-locked address generator and the vault transaction builder. The new page wires those functions together around a single mental model: I am the owner; I am deciding who receives what, and when it becomes available.

From inheritance intent to a distribution package



The plan remains editable until Build. Building creates an artifact; it does not broadcast it.

Product workflow. The Builder converts estate intent into a reviewable plan and, only after explicit confirmation, a half-signed distribution artifact.

3.1 Vault source and plan ledger

The workflow begins by loading an existing vault using the current Address, WIF, or Redeem Script flow. Until a vault is loaded, planning sections remain inert and the interface explains why. Once loaded, the sticky plan header shows the plan name, available balance, allocation percentage, unassigned remainder, reserved network fee, local draft status, and whether edits have diverged from the last build.

ACCOUNTING RULE Beneficiary outputs + return/change output + reserved network fee must equal the loaded vault balance. An unassigned remainder is returned to the vault; it is never silently converted into a miner fee.

3.2 One plan-wide release condition

The owner answers one consequential question: when should the inheritance plan take effect? A known date or block uses Bitcoin timing. An unknown event such as death or incapacity requires a monitoring arrangement. The interface keeps this plan-wide decision visually separate from the dates attached to individual beneficiary portions.

3.3 Beneficiaries and allocation schedules

Each beneficiary receives a color-coded card containing a local name, one beneficiary public key, a running subtotal, and one or more allocation rows. A row contains an unlock choice, an amount shown in both BTC and percentage, a derived read-only address, and a plain-language consequence sentence. Technical details such as the redeem script and shareable URL remain available behind a disclosure.

Choice	Owner sees	System produces
Immediately	This portion is available when the plan takes effect.	A non-time-locked beneficiary output.
On a date	This portion becomes available on the selected calendar date.	A CLTV-gated output derived from the beneficiary public key and date.
At a block	This portion becomes available around an estimated date at the selected height.	A block-height CLTV output; calendar timing is explicitly approximate.

When two portions for the same beneficiary use the same unlock condition, they resolve to the same address. The interface should offer to merge them. Every completed allocation row flows automatically into the transaction output list; the user never copy-pastes derived addresses between tools.

3.4 Timeline review and validation

A list view answers what each beneficiary receives. A timeline view answers when. The timeline marks the plan-wide release boundary, places immediate portions at that boundary, and highlights any beneficiary lock that falls before the release condition. The condition is not necessarily invalid, but it must be acknowledged because the portion cannot become effective before the transaction itself is distributed.

Before Build is enabled, the product validates that the vault is loaded, the release condition is set, all public keys and allocations are complete, amounts do not exceed the available balance, the entire balance is accounted for, and every derived address/script is internally consistent.

3.5 Build result and distribution package

Build creates the actual half-signed transaction from the reviewed plan. It does not broadcast the transaction and does not transfer ownership. The result is packaged per beneficiary with the beneficiary's portions, unlock schedule, transaction artifact, redeem scripts, shareable links, and future-facing instructions.

Known-date packages may be given to beneficiaries in advance because the transaction cannot confirm before its nLockTime. Monitored-mode material must follow the selected monitoring process and should not be presented as directly usable beneficiary material before the triggering event. In both modes, the package must explain that a later owner revision can invalidate an earlier artifact.

4. Protocol architecture and custody model

The base architecture uses a 2-of-3 threshold vault. The owner controls K1 and K2, preferably on independent devices with separate backups. The heir side holds K3. Any valid spend requires two distinct signatures. Because the owner holds two keys, the owner can act unilaterally. Because the heir side has only one distinct key, it cannot spend alone.

Figure 1 — Key Custody & Trust Topology

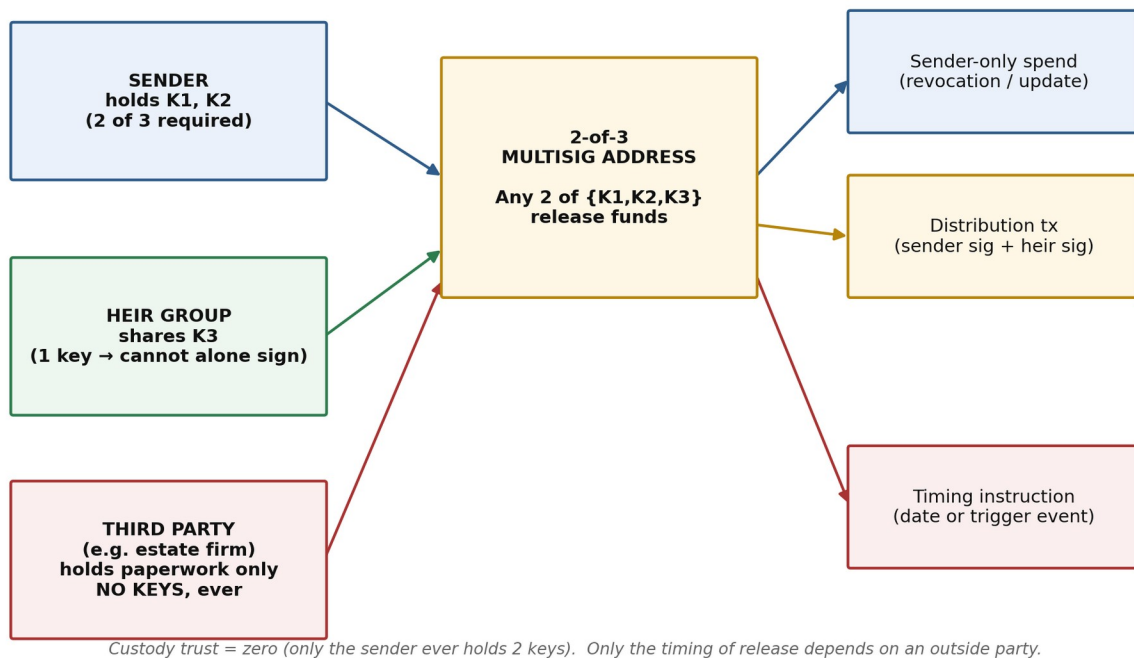


Figure 1. Key custody remains with the owner and heir side; an optional timing party holds instructions or an inert transaction artifact, never a working key combination.

4.1 The half-signed transaction

The owner constructs the full distribution transaction, fixing its inputs, outputs, beneficiary amounts, return/change, fee treatment, and optional locktime. The owner signs with one of K1 or K2 using a signature mode that commits to the intended transaction. One signature does not satisfy the 2-of-3 vault; the artifact cannot be mined or move funds by itself.

At the authorized time, the heir side adds K3 and broadcasts the completed transaction. The owner's original signature already commits to the beneficiary outputs, so the co-signer completes the approved transaction rather than inventing a new distribution. For multiple beneficiaries, a single completed transaction can fund all outputs simultaneously.

4.2 Vault key versus beneficiary receiving keys

Two key roles must remain distinct. K3 participates in spending the shared inheritance vault. Each beneficiary public key is used to derive that beneficiary's receiving output and any per-portion CLTV condition. Sharing K3 across an heir group does not require beneficiaries to share their final receiving keys or addresses.

4.3 Multiple heirs and collusion resistance

A naive design that gives each heir a different key inside a generic 2-of-(2+N) threshold lets any two heirs combine their keys and reach the threshold without the owner. The base Will-Wallet design avoids that construction by limiting the heir side to one distinct vault key, K3. Any number of copies still produces only one distinct signature and cannot satisfy 2-of-3 without an owner signature. This design choice must be reviewed carefully in implementation and explained in the beneficiary package.

5. Two independent timing layers

Inheritance timing contains two different questions. When may the distribution transaction confirm? When may each beneficiary spend the output they receive? Will-Wallet treats them as separate layers because Bitcoin enforces them through different mechanisms.

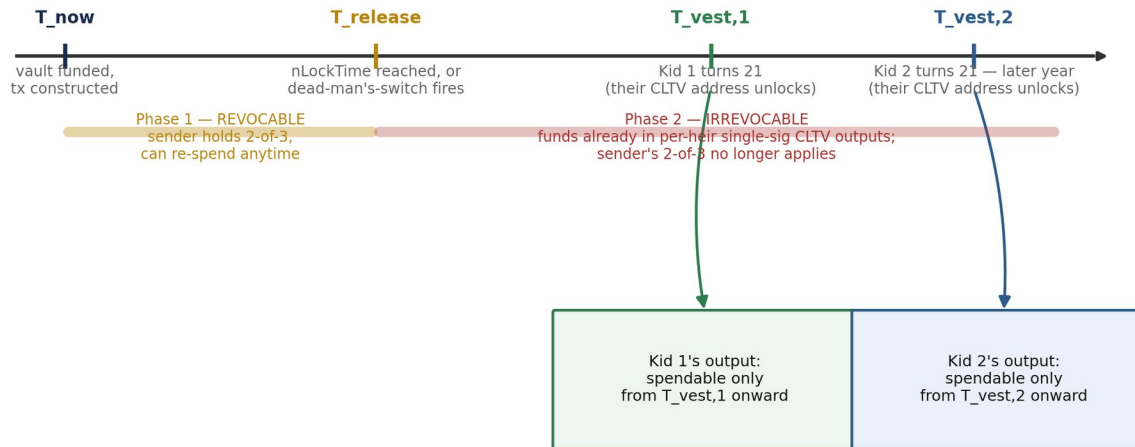
Figure 2 — Two-Stage Temporal Release (per-heir vesting)

Figure 2. Layer 1 controls distribution of the estate; Layer 2 controls when each beneficiary portion becomes spendable.

5.1 Layer 1 - distribution timing

In known-date mode, the half-signed distribution transaction carries an nLockTime expressed as either a block height or Unix timestamp. At least one input must use a sequence value below 0xFFFFFFFF; otherwise the transaction is treated as final and the locktime is disabled. Production code should enforce non-final sequence values as an invariant and cover the rule with automated tests. [3][4]

In monitored mode, no on-chain field can detect death or incapacity. The transaction artifact is released only after an external process observes the event or a check-in period expires. The monitoring party is trusted for timing, not custody.

5.2 Layer 2 - beneficiary vesting

Each beneficiary portion may pay to a CHECKLOCKTIMEVERIFY (CLTV) script derived from the beneficiary public key and a date or block height. Once the distribution transaction confirms, the output itself cannot be spent before that condition is met. Unlike nLockTime on one pre-built transaction, CLTV constrains every future spend of that output. [2]

One beneficiary may receive multiple outputs with different release dates, supporting staged distributions such as 25% at age 21, 25% at age 25, and the balance at age 30. Every extra installment increases transaction size and future operational complexity, so the product should discourage unnecessary granularity.

5.3 Time representation

Lock values below 500,000,000 are interpreted as block heights; values at or above that threshold are interpreted as Unix time. Calendar commitments should use timestamp form. Block-height schedules must be labeled approximate because block production is variable. Timestamp-based locktime is evaluated using Bitcoin's median-time-past rules rather than a wall-clock guarantee. [3][4]

6. Revocation, updating, and the irreversible boundary

A useful inheritance plan must be changeable. Families change, keys are replaced, balances move, and the intended timing may be revised. In Will-Wallet, revocation is not a special off-chain permission. It follows from the owner's control of the vault UTXO.

Figure 3 — Revocability State Machine

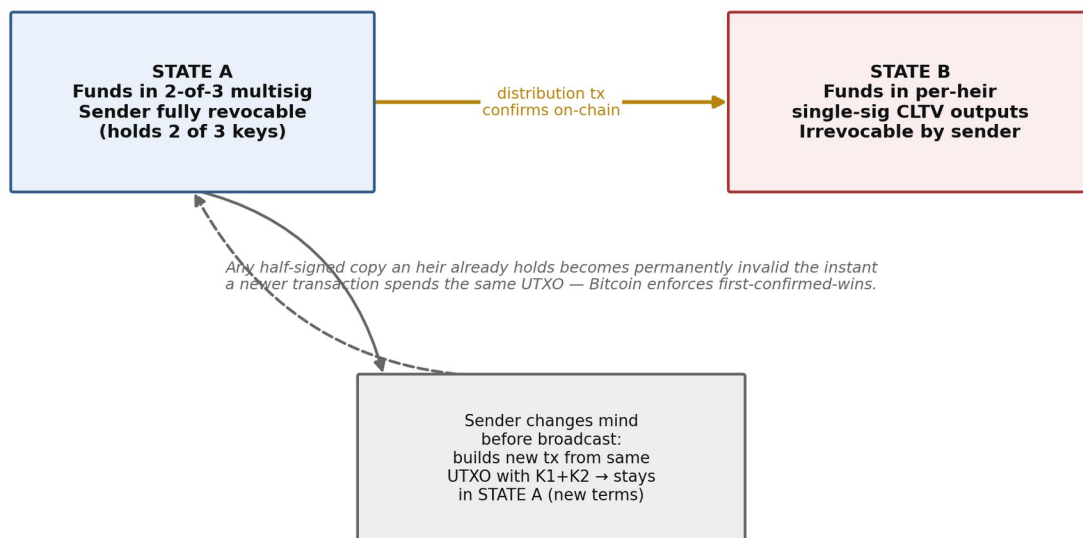


Figure 3. The distribution transaction's on-chain confirmation is the single irreversible transition in the base protocol.

6.1 How an update invalidates an old plan

Before the distribution transaction confirms, the owner can create and confirm a new owner-authorized transaction spending the same vault output with K1 and K2. Once that newer spend confirms, every earlier half-signed transaction referring to the spent output becomes invalid under Bitcoin consensus. Copies cannot be repaired or broadcast successfully because the input no longer exists as an unspent output.

The Builder should show whether a draft differs from the last built artifact, but version history, remote recall of files, and beneficiary redistribution are separate operational concerns. Will-Wallet cannot delete a file already shared with another person; it makes that file economically unusable by spending the referenced vault output first.

6.2 The race at release

Near the planned release time, an owner revision and a beneficiary broadcast may compete. The result depends on which conflicting transaction confirms first. This is an inherent property of on-chain revocation, not a user-interface issue. Owners should revise materially before a known release date, and monitored plans should define hold periods, escalation, and confirmation procedures.

IRREVERSIBLE BOUNDARY After the distribution transaction confirms, the owner's 2-of-3 vault authority no longer controls the beneficiary outputs. If those outputs are CLTV-gated, they remain locked until their own dates, but the owner cannot call them back.

7. Security, trust, and operational controls

7.1 Trust is minimized by dimension

The word trustless is too broad for inheritance. Will-Wallet evaluates custody, release timing, beneficiary vesting, and software correctness separately.

Trust dimension	Who could violate it	Protocol position
Custody	A party holding a threshold of vault keys	Provider custody is eliminated in the base model; only the owner holds two vault keys.
Known-date release	No external party	Enforced by nLockTime and input sequence rules on the prepared transaction.
Unknown-event release	Monitoring service or institutional executor	Irreducible timing trust; limited to detecting the event and delivering an inert artifact.
Beneficiary vesting	No external party after funding	Enforced by each CLTV output once the distribution confirms.
Software correctness	Builder, signing devices, libraries, user process	Not eliminated; requires independent review, deterministic tests, verification, and recovery procedures.

7.2 Threats and required mitigations

Risk	Failure mode	Control
Early release	Heir tries to act before the selected date.	Known-date nLockTime plus non-final sequence; monitored artifact withheld until trigger; CLTV where per-portion delay is required.
Redirected outputs	Co-signer attempts to change recipients or amounts.	Owner signature must commit to the complete intended transaction; verify transaction before signing and again before package export.
Heir collusion	Multiple heirs try to meet the vault threshold.	Base design exposes only one distinct heir-side vault key, not one threshold key per heir.
Stale fee market	A long-held transaction confirms slowly or not at all.	Define and test a fee-bumping strategy before production, including a CFPF-compatible output where appropriate; do not promise RBF unless signatures and policy support it.
Owner key loss	Owner cannot revise or recover the vault.	Independent hardware storage and tested backups for K1/K2; recovery design is an operational prerequisite.
Beneficiary key loss	A beneficiary cannot spend a released CLTV output.	Require beneficiary key verification and backup education; consider a separately reviewed recovery path for high-value plans.
Timing-party failure	Monitored release is early, late, or never delivered.	Dual contact paths, escalation, audit logs, grace periods, and a second institutional process; timing party still receives no spendable key threshold.
Artifact exposure	A half-signed transaction or script package leaks.	Treat packages as sensitive, encrypt at rest, minimize copies, and explain that exposure may change timing risk even when it does not create custody.

7.3 Verification and release controls

- Derive and display every output address from the beneficiary public key and selected lock condition.
- Require explicit confirmation of the vault input, destination outputs, BTC amounts, change, fee, locktime type, and sequence values before signing.
- Use a stable interchange format such as PSBT for hardware-wallet compatibility when the implementation is ready for that migration. [5]
- Test on regtest and signet/testnet workflows before mainnet use; never use example private keys from documentation.
- Perform independent cryptographic, application-security, and usability review before production release.
- Include a recovery rehearsal: owner revision, beneficiary completion, fee bump, and invalidation of an older artifact.

8. Privacy and beneficiary usability

An inheritance plan contains relationships, names, balances, dates, and potentially enough technical material to complete a future transaction. The product should minimize data collection by default and make local storage visible rather than implied.

8.1 Local-first plan data

- Beneficiary names are local labels for the owner; they are never embedded in the Bitcoin transaction or written to the blockchain.
- Draft plans auto-save on the device and clearly show when and where they were saved.
- Exported plan files and distribution packages are user-controlled artifacts. The UI should encourage encryption, offline backup, and careful version labeling.
- Public keys, addresses, transaction identifiers, amounts, and timing may become visible on-chain when the transactions confirm. The interface must not describe the plan as private once broadcast.

8.2 Instructions for a future non-technical recipient

A beneficiary package may be used years after it was created, possibly by a person who never used Bitcoin before. Its instructions should answer five questions without jargon:

1. What is this package and who prepared it?
2. When may it be used, and what condition controls that timing?
3. What must the beneficiary safeguard, including their own key and the redeem script?
4. How does the beneficiary verify that the package has not been superseded?
5. What exact steps, tools, and qualified support are needed to complete and broadcast the transaction?

The package should avoid presenting raw transaction hex as the primary experience. It may include the raw artifact for auditability, but the human-facing layer should use a clear status, amounts, dates, verification checks, and a step-by-step completion guide.

9. Institutional and commercial model

The commercial opportunity is strongest where the protocol cannot remove external information: determining an unknown real-world event. Estate-planning firms, trust institutions, and qualified professional services already verify death or incapacity and communicate with beneficiaries. Will-Wallet can add a non-custodial digital-asset artifact to that workflow without asking the institution to hold client Bitcoin keys.

Responsibility	Will-Wallet software	Estate / monitoring institution
Key generation and transaction construction	Provides the interface, derivation, validation, and artifact creation.	Does not generate or retain an owner spending threshold unless a separately reviewed model explicitly says so.
Custody	Never holds a working vault-key threshold.	Never holds a working vault-key threshold.
Known-date enforcement	Encodes and verifies Bitcoin timing rules.	No ongoing timing role required.
Unknown-event determination	Supports the configured workflow and artifact delivery controls.	Performs check-ins, death/incapacity verification, escalation, and authorized release.
Legal plan integration	Provides accurate technical descriptions and exportable records.	Coordinates wills, trusts, beneficiary records, client consent, and jurisdiction-specific compliance.

9.1 Revenue paths

- One-time plan setup and transaction-construction fee.
- Monitoring subscription only for unknown-event plans that require continuing operational service.
- Additional-beneficiary or multi-schedule service tier, reflecting greater setup and support complexity.
- Institutional licensing or white-label delivery for estate and trust firms.
- Referral or integration partnerships with qualified estate-planning professionals, subject to legal and ethical review.

POSITIONING The institution's value is verification, continuity, and guided delivery - not custody of Bitcoin. Marketing and contracts should preserve that boundary precisely.

10. Implementation path and acceptance criteria

The agreed product sequence starts with the smallest complete functional loop: one beneficiary public key, multiple unlock-and-amount rows, live-derived addresses, and automatic output population from an already loaded vault. Every later feature is an organization or repetition of that core.

Phase	Scope	Exit criteria
1. Functional core	Single beneficiary card; N allocation rows; live derived addresses; read-only output rollup.	Every completed row produces the correct address, amount, script, and transaction output without copy-paste.
2. Multi-beneficiary shell	Add/remove/duplicate cards, color identity, subtotals, remainder allocation.	No over-allocation; totals remain correct across edits, removals, and duplicated schedules.
3. Plan controls	Header ledger, release condition, local draft state, divergence from last build.	User can distinguish draft, built, broadcast, and confirmed states; balance equation always reconciles.
4. Review experience	List/timeline views, plain-language summary, technical disclosure, validation checklist.	Timeline catches pre-release beneficiary dates and same-date collisions; Build has a specific disabled reason.
5. Build and packages	Half-signed transaction generation; per-beneficiary package; save/print/export.	Artifact independently validates, contains no owner key, and carries usable future instructions.
6. Hardening	Automated protocol tests, hardware signing, fee strategy, secure storage, independent review.	Mainnet release gate approved by technical, security, legal, and operational reviewers.

10.1 Protocol test gates

- nLockTime is active because at least one input sequence is non-final; negative tests prove an all-final transaction is rejected by the Builder.
- Timestamp and block-height values are encoded and labeled correctly around the 500,000,000 boundary.
- Owner signatures commit to the intended outputs, amounts, fee handling, and locktime; the heir side cannot redirect funds.
- Every CLTV output is unspendable before its condition and spendable after it under the expected median-time-past/block rules.
- A confirmed owner revision invalidates every older artifact spending the same UTXO.
- Fee-bumping and long-delay transaction recovery are exercised under high-fee simulations.
- A multi-beneficiary transaction produces the same results shown in the plan ledger and each package.

10.2 Product test gates

- The owner never types a raw address, Unix timestamp, redeem script, key format, or fee amount in the primary plan flow.
- BTC and percentage remain synchronized, and entry beyond the unallocated remainder is prevented rather than merely flagged.
- Technical details remain available for audit without dominating the primary experience.
- Local-only labels and the consequences of export are explained at the point of use.
- Accessibility review covers color-independent beneficiary identification, keyboard operation, readable validation, and print output.
- Beneficiary packages pass moderated testing with non-technical users and remain understandable after a long time gap.

11. Limitations and legal positioning

Will-Wallet controls transaction validity and timing. It does not determine legal title, marital or community-property rights, creditor rights, probate obligations, fiduciary duties, tax basis, reporting, or whether a transfer satisfies a jurisdiction's inheritance formalities. The protocol should be integrated into conventional estate documents and reviewed by qualified professionals.

The model also cannot remove every technical risk. The owner may lose keys. A beneficiary may lose a receiving key. A monitoring party may act early or late. A long-held transaction may face an unfavorable fee market. Software may contain defects. On-chain privacy is limited. A plan may become obsolete as wallets, policies, or Bitcoin standards evolve.

For those reasons, production deployment should include periodic plan review, key-recovery testing, artifact revalidation, beneficiary contact verification, software maintenance, and a documented migration path. No document should promise that a plan can be created once and ignored indefinitely.

NOT A WILL OR TRUST Will-Wallet is execution infrastructure for digital assets. A legally valid estate plan determines who should receive property; Will-Wallet helps create a technically actionable path for the Bitcoin component.

12. Conclusion

The Bitcoin inheritance problem is not solved by hiding a seed phrase in a drawer or transferring present-day control to a custodian. It is solved by designing a future spending path that is authorized in advance, constrained by the owner's intent, understandable to the people who must use it, and revocable until the moment the owner deliberately gives up control.

Will-Wallet provides that structure with standard Bitcoin primitives: threshold control for the vault, a half-signed transaction for pre-authorization, nLockTime for a known distribution date, and CHECKLOCKTIMEVERIFY for beneficiary-specific vesting. Where the desired trigger is a real-world event, the remaining trust is narrowed to event detection and artifact delivery rather than control of funds.

The Inheritance Plan Builder turns those primitives into a calm planning experience. It asks who, how much, and when; keeps the plan editable; shows the economic consequences; and produces beneficiary packages only after explicit review. The result is not merely a transaction builder. It is a bridge between an estate-planning decision made today and a secure, executable transfer that may occur many years later.

VISION Make Bitcoin inheritance as deliberate as a will, as enforceable as a transaction, and as non-custodial as Bitcoin itself.

Appendix A. Technical requirements

Area	Requirement
Vault	Base model: 2-of-3 P2WSH-compatible threshold with owner K1/K2 and heir-side K3; exact production script policy subject to implementation review.
Owner signature	Must commit to the complete intended distribution transaction; unsupported or flexible sighash modes are not permitted in the inheritance flow.
Layer 1 timing	nLockTime as block height or Unix timestamp with at least one non-final input sequence; median-time-past semantics accounted for.
Layer 2 timing	Per-output CLTV scripts derived from beneficiary public keys; spending transaction must satisfy CLTV's locktime type and sequence requirements.
Transaction exchange	Current implementation may use raw transaction artifacts; PSBT is the preferred future interchange path for broader signer compatibility.
Fees	No manual fee field in the primary plan flow; fee estimate, change, and fee-bump path must be visible in review and tested for long-held transactions.
Draft data	Local autosave; explicit export; names and plan labels never written on-chain.
Verification	Independent output/script derivation, reproducible transaction decoding, test-network rehearsal, and production security review.

Appendix B. References

- [1] **Will-Wallet: A Peer-to-Peer Trustless Custodian Solution.** Yilak Kidane, 2018. Legacy project paper supplied with this white paper.
- [2] **BIP 65 - OP_CHECKLOCKTIMEVERIFY.** <https://github.com/bitcoin/bips/blob/master/bip-0065.mediawiki>
- [3] **BIP 113 - Median time-past for lock-time constraints.**
<https://github.com/bitcoin/bips/blob/master/bip-0113.mediawiki>
- [4] **Bitcoin Developer Guide - Locktime and sequence number.**
<https://developer.bitcoin.org/devguide/transactions.html#locktime-and-sequence-number>
- [5] **BIP 174 - Partially Signed Bitcoin Transaction format.**
<https://github.com/bitcoin/bips/blob/master/bip-0174.mediawiki>
- [6] **Will-Wallet: A Non-Custodial, Time-Conditional Protocol for Digital Asset Inheritance.** Yilak Kidane, Technical Paper - 2026 Revision. Supplied source document.
- [7] **Interface Design Brief v2 - Inheritance Plan Builder.** Will-Wallet, 2026. Agreed pre-build interface specification supplied with this white paper.